

Multi-Source Candidate Data Transformer — Technical Design

Candidate: Malay Shikhar Soni **Stack:** Node.js + TypeScript

Problem framing

Candidate data arrives from multiple sources with different shapes, reliability, and gaps. The goal: one deterministic, explainable pipeline producing a single canonical profile per candidate, with full visibility into where each value came from and how confident we are. Wrong-but-confident is the failure mode to design against, not missing data.

Pipeline

```
ingest -> extract -> normalize -> merge -> confidence -> project -> validate -> emit
```

- **ingest** — read each source (Recruiter CSV, GitHub profile via REST API). A missing file, empty file, or failed API call produces zero facts for that source, never a crash.
- **extract** — map each source into `RawFact { field, rawValue, source, sourceMethod }`. Downstream *processing logic* is source-agnostic, but `source/sourceMethod` travel with every value — this is what builds the final `provenance` array.
- **normalize** — pure per-field functions (phone → E.164 via `libphonenumber-js`, dates → `YYYY-MM`, country → ISO-3166 alpha-2, skills → canonical name via synonym lookup). A normalizer that can't parse its input returns `null`, never a guess.
- **merge** — group facts by match key (`email` , falling back to `name + company`), resolve each field via the precedence policy below. `candidate_id = sha256(matchKey)` — deterministic and traceable like everything else.
- **confidence** — per-field score, independent of which value won: CSV base `0.9` , GitHub-derived `0.6` ; `+0.1` if a second source agrees; `×0.5` if normalization failed. Clamped `[0,1]` .
- **project** — apply runtime config (field subset, `from` renames, per-field normalize override, provenance/confidence toggles, `on_missing`) to the canonical record without mutating it. Array fields (e.g. `skills[]`) support a `[]` path segment (`"skills[].name"`) mapping the rule over every element.
- **validate** — schema-check (Zod) before emit. Missing required fields trigger the configured `on_missing` (`null/omit/error`). Failures are caught per-candidate — one bad record never aborts a batch.

Canonical schema & formats

As specified: `candidate_id` , `full_name` , `emails[]` , `phones[]` (E.164), `location {city, region, country}` (ISO-3166 alpha-2), `links {linkedin, github, portfolio, other[]}` , `headline` , `years_experience` , `skills[{name, confidence, sources[]}]` , `experience[]` (YYYY-MM), `education[]` , `provenance[{field, source, method}]` , `overall_confidence` .

Merge / conflict-resolution policy

1. Precedence per field: **structured before unstructured** (CSV before GitHub-derived).
2. A higher-precedence source wins only if its value is non-null **and passes normalization** — a malformed CSV phone loses to a valid GitHub-derived one. This is what actually defeats “wrong but confident.”
3. Conflicting valid values: higher precedence wins, but the conflict is recorded in provenance (`method: "precedence_override_conflict"`), never silently dropped.
4. **Known limitation:** `name + company` fallback can collide for distinct people (e.g. two “John Smith”s, same company). Accepted at this scope — no fuzzy resolution attempted; documented rather than silently risked.

Scale & robustness

GitHub calls use an authenticated token (5,000 req/hr vs. 60 unauthenticated) with bounded concurrency; CSV is streamed, not fully buffered. At “thousands of candidates,” per-record failures (bad row, 404 profile) are caught and logged without aborting the batch.

Runtime custom-output config

Applied entirely in the **project** stage, after the canonical record is built. Canonical model and projection are decoupled: a new output shape needs only a new config object, no changes to ingest/normalize/merge. Output is schema-validated post-projection regardless of config.

Edge cases handled

1. Missing source entirely (blank GitHub URL, 404) — contributes zero facts, pipeline continues.
2. Conflicting non-null values (CSV name vs. GitHub display name) — resolved by precedence, flagged in provenance.
3. Malformed structured data (invalid CSV phone) — normalizer returns `null` , never fabricates.
4. Sparse unstructured source (GitHub profile with no bio/repos/links) — contributes only what exists.
5. Custom config requests a field neither source provides, `on_missing: "error"` — fails explicitly with a reason, not a silent partial record.

Deliberately out of scope

- `years_experience` / `education[]` — unreachable with CSV + GitHub; never fabricated, resolve to `null` /empty. Would need a 3rd source (resume/LinkedIn).
- Fuzzy/probabilistic entity resolution — matching is deterministic only (see known limitation above).
- LinkedIn scraping, resume parsing — not implemented; the `RawFact` + per-source extractor shape makes either a contained extension, not a redesign.
- LLM/agent-based extraction or matching — rejected deliberately: conflicts with “same input → same output,” and adds an unexplainable layer to a problem with a clean rule-based solution.